model/03/spec/support/size_matcher.rb
```
RSpec::Matchers.define :be_of_size do |expected|
  match do |actual|
    actual.size == expected
  end

  description do
    "have tasks totaling #{expected} points"
  end

  failure_message do |actual|
    "expected project #{actual.name} to have size #{expected}, was #{actual}"
  end

  failure_message_when_negated do |actual|
    "expected project #{actual.name} not to have size #{expected}, but it did"
  end
end
```

Three different customizations are used here: description takes no arguments and returns a string used by RSpec formatters that display the matcher's description. The other two are displayed by RSpec when the matcher fails, with failure_message being printed when a to match fails, and failure_message_on_negation being printed when a not_to match fails. You can also adjust the failure message by calling the method diffable in the matcher block—no arguments, no block. Effectively, you're declaring the matcher to be diffable, which means that on failure the expected and actual arguments are displayed in a diff format. RSpec uses this in the built-in string and array matchers.

If you want to be able to chain additional methods after the initial matcher to specify further arguments, then you use the chain method inside the matcher block:

model/04/spec/support/size_matcher.rb
```
RSpec::Matchers.define :be_of_size do |expected|
  match do |actual|
    size_to_check = @incomplete ? actual.remaining_size : actual.size
    size_to_check == expected
  end

  description do
    "have tasks totaling #{expected} points"
  end

  failure_message do |actual|
    "expected project #{actual.name} to have size #{expected}, was #{actual}"
  end

  failure_message_when_negated do |actual|
    "expected project #{actual.name} not to have size #{expected}, but it did"
  end
```